

# RELACIÓ ENTRE ALGORITMES RECURSIUS i ALGORITMES ITERATius

(i la seva verificació).

# RECURSIVITAT FINAL.

```
int fact (int n)
```

```
// Pre: cont
```

```
if (n == 0) f = 1;
```

```
else { // n > 1
```

```
    f = fact(n-1);
```

```
    f = f * n; }
```

```
return f;
```

```
Post: f = n!
```

Aquest programa és fàcilment verificable:

Case base:  $f = 1 \Rightarrow f = 1! = 1 \checkmark$

Case rec: (a)  $n-1 \neq n > 1$

(b)  $H.I: f = n-1!$

(c)  $f = n-1! \cdot n \Rightarrow f = n! \checkmark$

(d)  $fiba: n.$

Però no és recursiu fins

(fem  $f = f * n$  després de la crida rec.).

## Comentari sobre la funció FITA(I)

Hi pot haver diferents maneres de demostrar que una funció recursiva (iterativa) acaba.

Una és definir una funció:

$f: \mathbb{P} \rightarrow \mathbb{N}$  en  $\mathbb{P}$  són els paràmetres que canvien.

tot que per a tota crida recursiva

$\text{crida}(p; P)$

$\vdots$

$\text{crida}(h(p))$

tenim que:

$$f(p) > f(h(p))$$

Comentari sobre la funció FITA (II).

Què vol dir això?

$\text{crida}(p; P)$



$f(p) \in \mathbb{N}$

$\text{crida}(\underbrace{h(p)}_i)$

$\succ f(h(p)) \in \mathbb{N}$

com que  $f: \mathbb{P} \rightarrow \mathbb{N}$  decreix estrictament,  
arribarà un punt en què  $f(p) = 0$  i NO PODRÀ  
decreïxer més  $\Rightarrow$  la recursivitat acaba.

## Comentari sobre la funció FITA (III)

Dit altament:

(1) Ens assegurem que sempre decreixem;

$$f(p) > f(h(p))$$

(2) Posem un límit al decreixement:

Com que  $f: P \mapsto \mathbb{N}$

$$f(p) \geq \phi$$

# PLEGAT / DESPLEGAT

Volem obtenir una funció recursiva final (ja veurem després perquè).

El que calcula la funció anterior és:

$$\text{fact}(n) = \underbrace{\text{fact}(n-1)} * n \quad \text{(cas recursiu)}.$$

Ho generalitzem en una funció:

$$\text{gfact}(y, w) = \text{fact}(y) * w \quad \text{(recursiva FINAL)}.$$

La ~~post~~ de  $\text{gfact}(y, w)$  serà:  $f = \text{fact}(y) * w \equiv (\text{post de fact})$

$$f = y! * w$$

# PLEGAT / DESPLEGAT

Com seria el cas base de  $gfact(y, w)$ ?

$$\begin{aligned} gfact(y, w) &= fact(y) * w && (\text{cas base: } y=0) \\ &= fact(0) * w && (\text{cas base de fact}) \\ &= 1 * w \end{aligned}$$

$$gfact(y, w) = w$$

```
int gfact(int y, int w) {  
    if (y == 0) f = w;  
}
```

# PLEGAT / DESPLEGAT

Com seca el cas recursiu? ( $y > 0$ )

$$\text{gfact}(y, w) = \text{fact}(y) * w$$

(definició de fact  
en el cas recursiu  $y > 0$ )

$$\text{DESPLEGAT} = (\text{fact}(y-1) * y) * w$$

associativitat

$$\text{PLEGAT} = \text{fact}(y-1) * (y * w)$$

definició de gfact:

$$\text{gfact}(y, w) = \text{gfact}(y-1, y * w)$$

$$\text{gfact}(1, 1) = \text{fact}(1) * 1$$

if ( $y > 0$ )

f = gfact(y-1, y\*w);



# PLEGAT / DESPLEGAT

El programa queda així:

```
int gfact(int y, int w)
// Pre: cond
if (y == 0) f = w;
else // y > 0
    f = gfact(y-1, y*w);
return f
// Post: f = y! * w
```

Per tant, per calcular  $n!$  femem la crida

$\text{gfact}(n, 1)$

# PLEGAT / DESPLEGAT

Cal VERIFICAR aquesta funció:

```
int gfact(int y, int w)
```

```
// Pre: cont
```

```
if (y == 0) f = w;
```

```
else // y > 0
```

```
    f = gfact(y-1, y*w);
```

```
return f
```

```
// Post: f = y! * w
```

Cas base: {PRE n CONT<sub>base</sub>}

f = w;

{POST}

{y = 0}

f = w;

{f = y! \* w} =

{f = 1! \* w} =

{f = w} } ✓

## PLEGAT / DESPLEGAT

Cal VERIFICAR aquesta funció:

```
int gfact(int y, int w)
// Pre: cond
if (y == 0) f = w;
else // y > 0
    f = gfact(y-1, y*w);
return f
// Post: f = y! * w
```

Cas recursiu:

```
{PRE n CONDrec}
    f = gfact(y-1, y*w);
{POST}
```

```
{y > 0}
    f = gfact(y-1, y*w);
{f = y!}
```

Però abans hem vist que:

$\{y > 0\}$

$f = \text{gfact}(y-1, y * w);$

H.I.  $\text{gfact}(y-1, y * w)$

$\equiv$

$\{f = y! * w\}$

$\text{fact}(y) * w$

$\equiv$

$\text{post de fact}$

$y! * w$

✓

# PLEGAT / DESPLEGAT

- (1) Hem pogut transformar una funció recursiva NO final a una de FINAL amb la tècnica del plegat / desplegat.
- (2) Això només ho podem fer si les operacions són associatives (la gran majoria ho són).
- (3) La verificació de la primera funció IMPLICA la verificació de la funció transformada.

Però la funció no és encara a Postcondició

constant



int

gfact(int y, int w)

// Pre: cond

if (y == 0) f = w;

else // y > 0

f = gfact(y-1, y \* w);

return f

// Post: f = y! \* w



varien a cada  
crida recursiva.

La post no depèn dels  
paràmetres que varien.

Per passar-ho a post constant, farem un

artifici:

```
int gfact(int y, int w, int n)
```

```
// Pre:  $y! * w = n!$ 
```

```
if (y == 0) f = w;
```

```
else // y > 0
```

```
f = gfact(y-1, y * w, n);
```

```
return f
```

```
// Post:  $f = n!$ 
```

④ modifiquem la post

ara, la post no depèn  
de les paraèmètres que  
volem.

② modifiquem la pre

③ afegim el paraèmètre  
a la crida rec

① afegim un paraèmètre

## CAS RECURSIV

PRE:  $\{y! * w = n!\}$

CRIDA RECURSIVA:

$$f = \text{gfact}(y-1, y * w, n);$$

LA POST anterior eno deia:

$$f = y! * w$$

$$= n! \quad (\text{ho diu la PRS!!})$$

- La "idea" d'aquest paràmetre que no varia és definir que volem calcular.
- La PRE eno garantix, en tot moment, que volem "acostant-nos" al resultat.



# CONSIDERATIONS (I)

Pass a recursive find:

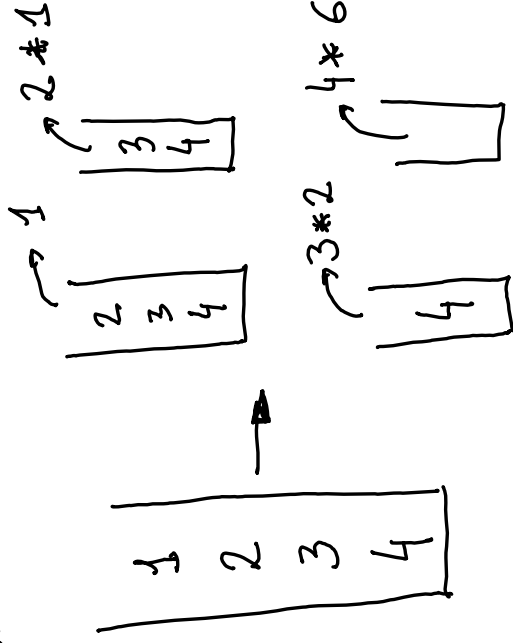
$$y = 0: f = 1$$

$$y > 0: f = \text{fact}(n-1)$$

$$f = f * n$$

resultat find

$$\begin{aligned} \text{fact}(4) &= \text{fact}(3) * 4 \\ &\quad \downarrow \\ &\text{fact}(2) * 3 \\ &\quad \downarrow \\ &\text{fact}(1) * 2 \\ &\quad \downarrow \\ &\text{fact}(0) = 1 \end{aligned}$$



## CONSIDERATIONS (II)

$y=0: f=w;$

$y>0: f=gfact(y-1, y*w);$

$$\begin{aligned} gfact(4, 1) &= gfact(3, 4*1) \\ &= gfact(2, 4*3) \\ &= gfact(1, 4*3*2) \\ &= gfact(0, 4*3*2*1) \\ &= 4*3*2*1 \end{aligned}$$

→ result to find

## CONSIDERACIONS (III)

És a dir, passar a recursivitat final  
vol dir arribar al cas base amb el  
resultat (gairebé) calculat.

Altresament, el que fem és "empilar" els  
valors i calcular el resultat quan  
tornem de les crides recursives.

## CONSIDERACIONS (IV)

I el  $\rho_{00}$  a POST constant?

El que fem és posar un paràmetre extra que verifiqui que a cada  $\rho_{00}$  extern correspon el valor desitjat.

És una mena de "testimoni".

I ara veurem com ens resulta útil.

De recursiu final

amb postcondició constant

a PROGRAMA ITERATIU.

## DE RECURSION

## A ITERATION

```
int gfact(int y, int w, int n)
// Pre: y! * w = n!
if (y == 0) f = w;
else // y > 0.
    f = gfact(y-1, y * w, n);
return f
// Post: f = n!
```

int gfact-it(int y, w, n)

// Pre: y! \* w = n!

while (y > 0) {

    y = y - 1

    w = y \* w

    f = w;

return f;

// Post: f = n!

cal tenir en compte que cridem

aquesta funció amb els valors

fact-it (int y, w, n)

|   |   |
|---|---|
| 4 | ↑ |
| n | 1 |

Podem inicialitzar aquestes variables  
a dins del codi.

## DE RECURSION

## → AN ITERATIVE

```
int gfact(int y, int w, int n)
```

```
// Pre:  $y! * w = n!$ 
```

```
if ( $y == 0$ )  $f = w$ ;
```

```
else //  $y > 0$ 
```

```
     $f = \text{gfact}(y-1, y * w, n)$ ;
```

```
    return  $f$ 
```

```
// Post:  $f = n!$ 
```

```
int gfact-it(int n)
```

```
// Pre: cent
```

```
     $y = n$ ;
```

```
     $w = 1$ ;
```

```
// INV:  $y! * w = n!$ 
```

```
    while ( $y > 0$ ) {
```

```
         $y = y - 1$ 
```

```
         $w = y * w$ 
```

```
    }
```

```
     $f = w$ ;
```

```
    return  $f$ ;
```

```
// Post:  $f = n!$ 
```



## DE RECURSIU

```
int gfact(int y, int w, int n);
```

```
// Pre:  $y! * w = n!$ 
```

```
if ( $y == 0$ )  $f = w$ ;
```

```
else //  $y > 0$ 
```

```
     $f = \text{gfact}(y-1, y * w, n)$ ;
```

```
    return  $f$ 
```

```
// Post:  $f = n!$ 
```

Cas base:  $\{y! * w = n! \wedge y = 0\}$

$f = w$

$\{f = n!\}$

(A)

Cas recursiv: La part de la crida recursiva  
és la mateixa que la de la funció  
iterativa.

## A ITERATIVU

```
int gfact-it(int n)
```

```
// Pre: cent
```

```
     $y = n$ ;
```

```
     $w = 1$ ;
```

```
// INV:  $y! * w = n!$ 
```

```
    while ( $y > 0$ ) {
```

```
         $y = y - 1$ 
```

```
         $w = y * w$ 
```

```
    }
```

```
     $f = w$ ;
```

```
    return  $f$ ;
```

```
// Post:  $f = n!$ 
```

## DE RECURSIU

```
int gfact(int y, int w, int n)
```

```
// Pre:  $y! * w = n!$ 
```

```
if ( $y == 0$ )  $f = w$ ;
```

```
else //  $y > 0$ 
```

```
     $f = \text{gfact}(y-1, y * w, n)$ ;
```

```
    return  $f$ 
```

```
// Post:  $f = n!$ 
```

## A ITERATIVU

```
int gfact-it(int n)
```

```
// Pre: cent
```

```
     $y = n$ ;
```

```
     $w = 1$ ;
```

```
// INV:  $y! * w = n!$ 
```

```
    while ( $y > 0$ ) {
```

```
         $y = y - 1$ 
```

```
         $w = y * w$ 
```

```
    }
```

```
     $f = w$ ;
```

```
    return  $f$ ;
```

```
// Post:  $f = n!$ 
```

Case recursive:

```
{  $y! * w = n!$  ^  $y > 0$  }
```

```
 $y = y - 1$ 
```

```
 $w = y * w$ 
```

```
{  $(y-1)! * y * w = n!$  }
```

(B)

## ITERATION

```
int fact-it (int n)
```

```
// Pre: cent
```

```
y = n;
```

```
w = 1;
```

```
// INV:  $y! * w = n!$ 
```

```
while (y > 0) {
```

```
    y = y - 1
```

```
    w = y * w
```

```
}
```

```
f = w;
```

```
return f;
```

```
// Post:  $f = n!$ 
```

Què cal verificar en un algoritme iteratiu?

(1) {PRE} invariants  $\Rightarrow$  INV

{cert}

y = n;

w = 1;

{  $y! * w = n!$  } ✓

(2) {INV} BUCLE  $\Rightarrow$  {INV}

{  $y! * w = n!$  }

y = y - 1

w = y \* w

(B)

{  $y! * w = n!$  }  $\Rightarrow$  {  $(y-1)! * w = n!$  }

ja ho tenim ✓

## ITERATIO

Què cal verificar en un algoritme iteratiu?

```
int gfact-it(int n)
```

```
// Pre: cent
```

```
    y = n;
```

```
    w = 1;
```

```
    // INV: y! * w = n!
```

```
    while (y > 0) {
```

```
        y = y - 1
```

```
        w = y * w
```

```
    }
```

```
    f = w;
```

```
    return f;
```

```
// Post: f = n!
```

(3) { INV } instruccions  $\Rightarrow$  POST  
      ^  
      { condició } final

{ y! \* w = n! ^ y = 0 }

(A)

f = w;

{ f = n! }

ja hem arribat ✓

(4)

La funció de fita de la funció iterativa és la mateixa que la funció recursiva.

## RECAPITULACIÓ (I)

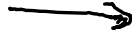
Funció recursiva no final



Plegat / Desplegat (si les operacions són associatives).

Funció recursiva final

(ja verificada, si hem verificat l'anterior).



Afegim paràmetres que no varien.

Funció recursiva final amb postcondició constant



Funció iterativa

(ja verificada, si hem verificat la funció recursiva).

COMPTA!!

## RECAPITULACIÓ (II).

1. En molts casos podem passar d'una funció recursiva a una iterativa.
2. Normalment no és aquesta la manera d'escriure una funció iterativa.
3. El que hem vist:
  - (a) la relació entre recursivitat i iterativitat.
  - (b) hem aplicat de manera "natural" la inducció en algorismes recursius.
  - (c) la relació recursiu  $\rightarrow$  iteratiu permet aplicar (transitivament) la inducció a algorismes iteratius.

## RECAPITULACIÓ (III)

- Tenim una manera automàtica de transformar i verificar una funció recursiva en una d'iterativa.
- La iteració és una manera de computar una funció recursiva.
- La base de la demostració de la correcció és la INDUCCIÓ que apliquem a funcions ... recursives.

